MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA

TECHNICAL UNIVERSITY OF MOLDOVA

Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automatic

REPORT ON LABORATORY WORK No. 3

Course: Cryptography and Security

Done by: student, group FAF-243
Andrei Elciscev

Checked by:
Zaica Maia

Chișinău, 2026

Contents

1 Introduction **2**

2 Objectives **2**

3 Theoretical Background **2**

3.1 Polyalphabetic Ciphers 2

3.2 The Vigenère Cipher 3

3.2.1 Example 3

3.3 The Playfair Cipher 4

3.4 Alphabet Used in the Implementation 4

3.5 Construction of the Playfair Matrix 4

3.6 Preparation of Digraphs 5

3.7 Encryption Rules 5

3.7.1 Rule 1: Same Row 5

3.7.2 Rule 2: Same Column 5

3.7.3 Rule 3: Rectangle 5

3.8 Decryption Rules 6

4 Implementation **6**

4.1 Development Environment 6

4.2 Matrix Construction 6

4.3 Character Validation 6

4.4 Digraph Preparation 7

4.5 Encryption and Decryption 7

4.6 Console Interface 7

5 Testing and Results **7**

5.1 Test 1: Playfair Encryption 7

5.2 Test 2: Playfair Decryption 8

5.3 Test 3: Invalid Operation 9

5.4 Test 4: Invalid Key 9

5.5 Test 5: Invalid Plaintext Character 10

5.6 Test 6: Odd-Length Ciphertext 10

5.7 Testing Summary 11

6 Conclusions **12**

1 Introduction

Cryptography is the study of techniques used to protect information by transforming readable messages into forms that cannot be easily understood by unauthorized individuals. Encryption and decryption are fundamental operations in cryptographic systems.

Classical cryptography includes several substitution techniques. A monoalphabetic substitution cipher replaces each plaintext character with a corresponding ciphertext character using a fixed substitution. A weakness of such ciphers is that the same plaintext character always produces the same ciphertext character, making them vulnerable to frequency analysis.

Polyalphabetic ciphers improve upon this approach by using multiple substitutions. The substitution applied to a character depends on its position in the message or on a repeating key. This reduces the direct relationship between plaintext letters and ciphertext letters.

This laboratory work studies the Vigenère cipher and the Playfair cipher. The practical implementation focuses on the Playfair cipher adapted to a Romanian character set and developed in C#.

2 Objectives

The objectives of this laboratory work are:

- To study the principles of polyalphabetic encryption.
- To understand the operation of the Vigenère cipher.
- To study the encryption and decryption rules of the Playfair cipher.
- To adapt the Playfair cipher to a 6-by-6 matrix.
- To implement encryption and decryption in C#.
- To validate user input and handle invalid data.
- To test the application using different input scenarios.

3 Theoretical Background

3.1 Polyalphabetic Ciphers

A polyalphabetic substitution cipher uses more than one substitution alphabet. Unlike a monoalphabetic cipher, the same plaintext letter may be encrypted into different ciphertext letters depending on its position and the key.

The Vigenère cipher is a well-known example. It uses a repeating keyword to determine the shift applied to each plaintext character.

The Playfair cipher is a digraph substitution cipher. Instead of encrypting individual characters, it encrypts pairs of characters using their positions in a matrix. It is traditionally classified as a digraphic substitution cipher and is commonly studied alongside classical polyalphabetic methods.

3.2 The Vigenère Cipher

The Vigenère cipher uses a keyword that is repeated until it matches the length of the plaintext. Each letter is represented by a numerical value, where $A = 0$, $B = 1$, and so on, through $Z = 25$.

The encryption formula is:

$$C_i = (P_i + K_i) \bmod 26$$

The decryption formula is:

$$P_i = (C_i - K_i + 26) \bmod 26$$

where:

- P_i is the numerical value of the plaintext character;
- K_i is the numerical value of the corresponding key character;
- C_i is the numerical value of the ciphertext character;
- i is the character position.

3.2.1 Example

Consider the plaintext:

`PERASPERAADASTRA`

The key is repeated to match the plaintext length:

`SUPERSUPERSUPERS`

Applying the Vigenère encryption formula produces:

`HYGEJHYGERVUHXIS`

The encryption process adds the corresponding plaintext and key values modulo 26. Decryption reverses this operation to recover the original plaintext.

3.3 The Playfair Cipher

The Playfair cipher was introduced in the nineteenth century and is associated with Charles Wheatstone and Lord Playfair. It encrypts pairs of letters, called digraphs, rather than individual characters.

The cipher uses a square matrix constructed from a keyword. Repeated letters in the key are removed, and the remaining alphabet letters are appended in their standard order.

In the traditional English version, the matrix is 5-by-5, and I and J are usually combined because the English alphabet contains 26 letters. This implementation uses a 6-by-6 matrix to accommodate the 31-character alphabet configured in the program.

3.4 Alphabet Used in the Implementation

The implementation defines the following character set:

A Ă Â B C D E F G H I Î J K L M N O P Q R S Ș T Ț U V W X Y Z

This set contains 31 characters. A 6-by-6 matrix has 36 cells, so five additional cells remain after all 31 letters have been placed. The implementation fills these cells with digits.

The digits are not accepted as plaintext or ciphertext characters. They are used only to complete the matrix.

3.5 Construction of the Playfair Matrix

The matrix is constructed according to the following procedure:

1. Convert the keyword to uppercase.
2. Validate the keyword against the permitted character set.
3. Add the distinct characters of the keyword to the matrix in their original order.
4. Append the remaining characters of the configured alphabet in their defined order.
5. Fill the remaining cells with digits.

For example, if the keyword contains repeated characters, each character is inserted only once. Characters already placed in the matrix are skipped when the rest of the alphabet is processed.

The resulting matrix is used for both encryption and decryption. Therefore, the same keyword must be provided when decrypting a message.

3.6 Preparation of Digraphs

Before encryption, the plaintext is converted to uppercase and whitespace is removed. The remaining characters are validated.

The plaintext is then divided into pairs according to these rules:

- If two consecutive characters are different, they form a digraph.
- If two consecutive characters are identical, a filler character is inserted after the first character, and the second character is processed again.
- If a single character remains at the end, a filler character is appended to complete the final pair.

The implementation uses `X` as the default filler character. If the character being processed is `X`, the program uses `Q` instead.

These rules ensure that the prepared plaintext contains an even number of characters and that no digraph contains two identical characters.

3.7 Encryption Rules

Let the positions of the two characters in a digraph be:

$$A = (r_1, c_1), \qquad B = (r_2, c_2)$$

The encryption procedure follows three rules.

3.7.1 Rule 1: Same Row

If both characters are in the same row, each character is replaced by the character immediately to its right. If a character is at the last column, the operation wraps around to the first column.

3.7.2 Rule 2: Same Column

If both characters are in the same column, each character is replaced by the character immediately below it. If a character is at the last row, the operation wraps around to the first row.

3.7.3 Rule 3: Rectangle

If the characters are in different rows and different columns, they form opposite corners of a rectangle. Each character is replaced by the character in its own row and the other character's column.

The resulting pair is appended to the ciphertext.

3.8 Decryption Rules

Decryption reverses the encryption operations.

- **Same row:** move one position to the left, wrapping around at the beginning of the row.
- **Same column:** move one position upward, wrapping around at the top of the column.
- **Rectangle:** exchange the column positions using the same rectangle rule.

The ciphertext must contain an even number of valid characters. After decryption, filler characters may remain in the output. The program does not automatically remove them because a filler character may also occur naturally in the original plaintext.

4 Implementation

4.1 Development Environment

The application was implemented in C# using the .NET environment. The program uses a console interface that allows the user to choose an operation, enter a key, and provide the text to process.

The implementation is divided into two main components:

- `PlayfairCipher`: contains the matrix construction, input validation, digraph preparation, encryption, decryption, and matrix display logic.
- `Program`: manages user interaction and displays the results or validation errors.

4.2 Matrix Construction

The `BuildMatrix` method creates a 6-by-6 character matrix. It first inserts the distinct valid characters of the key. It then appends the remaining characters from the configured alphabet.

A set is used to track which characters have already been placed. This prevents duplicate characters from appearing in the matrix.

4.3 Character Validation

The `IsAllowedLetter` method checks whether a character belongs to the configured alphabet. This validation is used when processing the key, plaintext, and ciphertext.

If the user provides an unsupported character, the program displays an error message and lists the permitted characters.

4.4 Digraph Preparation

The `PrepareDigraphs` method converts the input into uppercase characters, removes whitespace, and validates every remaining character. It then creates character pairs while applying the filler-character rules.

If the input is empty or contains an invalid character, the method returns an error message rather than continuing with encryption.

4.5 Encryption and Decryption

The `Encrypt` method builds the matrix, prepares the plaintext digraphs, finds the positions of the characters, and applies the appropriate Playfair rule to each pair.

The `Decrypt` method builds the same matrix and validates the ciphertext. It checks that the ciphertext length is even before processing each pair using the inverse rules.

Both methods return the resulting text as a string.

4.6 Console Interface

The main program displays the permitted alphabet and asks the user to choose between encryption and decryption.

The program also validates the operation and checks that the key contains at least seven characters. It then requests the appropriate plaintext or ciphertext and displays the result together with the matrix used.

During decryption, the program informs the user that filler characters may need to be removed manually and that spaces must be restored separately.

5 Testing and Results

Testing is necessary to verify that the application performs encryption and decryption correctly and handles invalid input appropriately.

The following tests are intended to demonstrate the main functionality and input-validation behavior of the program.

5.1 Test 1: Playfair Encryption

The first test demonstrates encryption with a valid key and plaintext. The program prepares the plaintext digraphs, encrypts them according to the Playfair rules, and displays the ciphertext and matrix.

```
Playfair cipher (Romanian alphabet, 31 letters)
Allowed letters:
A Ă Â B C D E F G H I Î J K L M N O P Q R S Ș T Ț U V X Y Z

Choose operation:
  E - encrypt
  D - decrypt
  X - exit
Operation: e
Key (minimum 7 letters): abacustech
Plaintext (letters only): bunĂ ziua

Ciphertext: CSOHȚLSB

Matrix used:
A B C U S T
E H Ă Â D F
G I Î J K L
M N O P Q R
Ș Ț V X Y Z
```

Figure 1: Playfair cipher encryption

5.2 Test 2: Playfair Decryption

The second test demonstrates decryption using the same key that was used for encryption. The program validates the ciphertext, processes each pair, and displays the decrypted result and matrix.

The decrypted output may contain filler characters inserted during plaintext preparation. Their removal requires contextual interpretation.

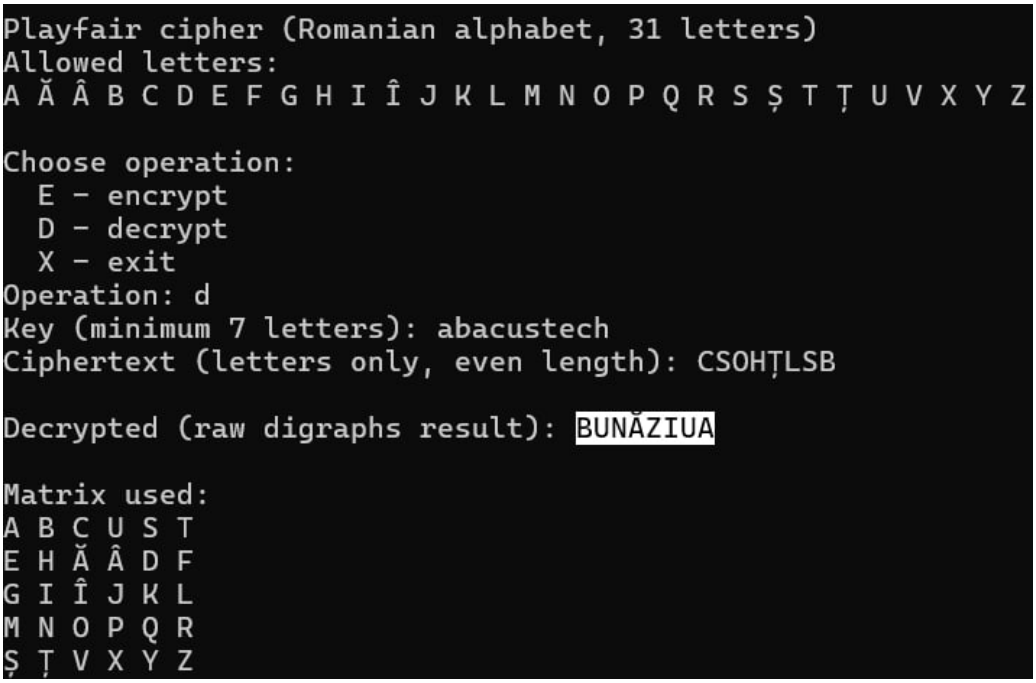


Figure 2: Playfair cipher decryption

5.3 Test 3: Invalid Operation

This test checks the behavior of the application when the user enters an operation other than `E` or `D`. The program displays an error message and terminates the current operation.

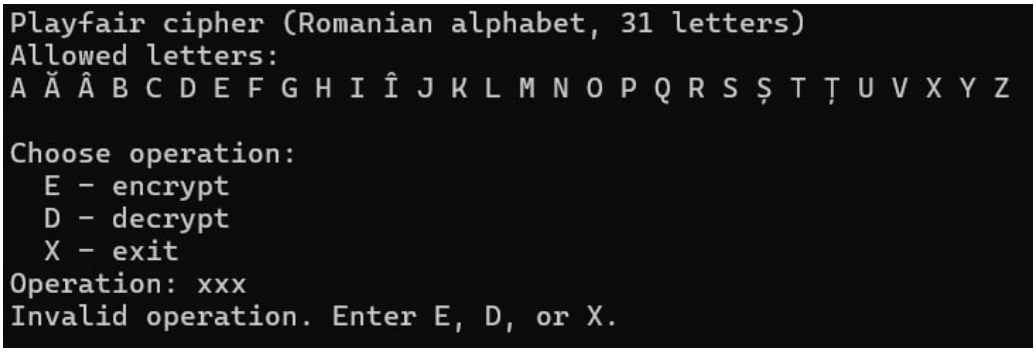


Figure 3: Validation of an invalid operation

5.4 Test 4: Invalid Key

This test demonstrates key validation. A key shorter than seven characters is rejected. The program also checks that all characters in the key belong to the configured alphabet.

```
Playfair cipher (Romanian alphabet, 31 letters)
Allowed letters:
A Ă Â B C D E F G H I Î J K L M N O P Q R S Ș T Ț U V X Y Z

Choose operation:
  E - encrypt
  D - decrypt
  X - exit
Operation: e
Key (minimum 7 letters): nope
Key too short. Minimum length is 7 letters.
```

Figure 4: Validation of an invalid key

5.5 Test 5: Invalid Plaintext Character

This test checks that unsupported characters are rejected during plaintext preparation. The program displays an error message identifying the invalid character and listing the allowed characters.

```
Playfair cipher (Romanian alphabet, 31 letters)
Allowed letters:
A Ă Â B C D E F G H I Î J K L M N O P Q R S Ș T Ț U V X Y Z

Choose operation:
  E - encrypt
  D - decrypt
  X - exit
Operation: e
Key (minimum 7 letters): abacustech
Plaintext (letters only): v0lat1le
Error: Character '0' is not allowed. Allowed letters are: A Ă Â
```

Figure 5: Validation of an invalid plaintext character

5.6 Test 6: Odd-Length Ciphertext

This test examines how the application handles ciphertext containing an odd number of characters. Since the Playfair cipher processes text in pairs of characters (digraphs), the ciphertext must have an even length for decryption to proceed.

When the entered ciphertext contains an odd number of characters, the program automatically appends an additional character to complete the final digraph. This allows the decryption process to continue without terminating due to an odd-length input.

However, the appended character was not necessarily part of the original ciphertext. Consequently, the final decrypted output may contain an additional character or may differ from the original plaintext. The result must therefore be interpreted with care, and

the automatically appended character should be taken into account when analyzing the decrypted message.

This test demonstrates the application's handling of odd-length ciphertext and its ability to continue decryption after adjusting the input length. It also illustrates an important limitation: adding a character ensures that the input can be processed in pairs, but it does not guarantee that the resulting plaintext is correct.

```
Operation: e
Key (minimum 7 letters): abacustech
Plaintext (letters only): impar

Ciphertext: GNMUPZ

Matrix used:
A B C U S T
E H Ă Â D F
G I Î J K L
M N O P Q R
Ș Ț V X Y Z

Note: Reinsert spaces manually if needed.

Playfair cipher (Romanian alphabet, 31 letters)
Allowed letters:
A Ă Â B C D E F G H I Î J K L M N O P Q R S Ș T Ț U V X Y Z

Choose operation:
  E - encrypt
  D - decrypt
  X - exit
Operation: d
Key (minimum 7 letters): abacustech
Ciphertext (letters only, even length): GNMUPZ

Decrypted (raw digraphs result): IMPARX
```

Figure 6: Automatic padding of odd-length ciphertext before decryption

5.7 Testing Summary

The tests cover the two principal operations and several invalid-input conditions. They demonstrate the intended behavior of the console application, including matrix generation, character validation, digraph processing, encryption, and decryption.

Successful decryption should recover the prepared plaintext, which may differ from the original text because filler characters can be inserted during encryption. Spaces and original formatting are not preserved by the implementation.

6 Conclusions

During this laboratory work, the principles of classical polyalphabetic and digraphic encryption were studied, with particular attention to the Vigenère and Playfair ciphers.

The Playfair cipher was implemented in C# using a 6-by-6 matrix and the 31-character alphabet configured in the application. The program constructs the matrix from a keyword, removes duplicate key characters, prepares plaintext digraphs, and performs encryption and decryption according to the Playfair rules.

Input validation was also implemented to reject unsupported characters, invalid operations, keys that do not meet the minimum length requirement, and ciphertexts with an odd number of characters.

The tests demonstrate the intended behavior of the application under valid and invalid input conditions. The laboratory work provided practical experience with classical encryption algorithms, character processing, matrix manipulation, and validation in C#.

Although these ciphers are useful for understanding cryptographic principles, they are not suitable for protecting sensitive modern communications because their security is limited by known cryptanalytic techniques.